Nandam Josh Nihanth

23PA1A05H1

CSE-C

DAY-04

SALESFORCE DEVELOPMENT TRAINING

Part 1 – Think Before Coding

1. Why do users need a graphical interface?

A graphical user interface (GUI) allows users to interact with the application through buttons, forms, and menus instead of writing code.

It makes the system user friendly and impoves the performance of it.

2. Why can't users directly execute SOQL queries?

SOQL queries are used to retrieve data from Salesforce databases and should only be executed by authorized server-side logic.

It can run directly without having any security issues of the data.

3. Why is JavaScript required in LWC?

JavaScript provides the logic for Lightning Web Components. It manages variables, handles user interactions such as button clicks, performs calculations

4. What responsibilities belong to the UI?

The User Interface is responsible for displaying information, collecting user input, handling navigation, and providing an interactive experience through components like buttons, forms, and cards.

5. Which responsibilities should remain in Apex?

Apex handles business logic, database operations, SOQL queries, DML operations, validations, and communication with Salesforce records. Improves the security and maintaince of it

---

Part 2 – Understanding LWC(Lightning web components)

Lightning Web Components (LWC) is Salesforce's modern framework for building user interfaces using standard web technologies like HTML, JavaScript, and CSS.

Lightning Web Component Structure :-

Database = Bricks (stores data)

Apex = Electric wiring (business logic)

LWC = Paint, doors, windows, furniture (what users actually see)

Every Lightning Web Component consists of three primary files:

- HTML File – Defines the structure and layout of the component, including text, buttons, and other visual elements.

  In this we can create the new components that is used for the html pages like buttons , persons data, images ,information and soon

- JavaScript File – Contains the component logic, variables, methods, and event handling.

  In this we can write the entire logic in the form of the javascript and that should have the logic, variales,events, communication between the methods in the javascript.

- Meta XML File – Specifies where the component can be used and makes it available in Lightning App Builder.

  The xml file we can used to visible in the salesforce developer platdorm that should if it value was true and also based on the salesforce version of it and that should be creating the componets like the homepage, apppage and desktop page.

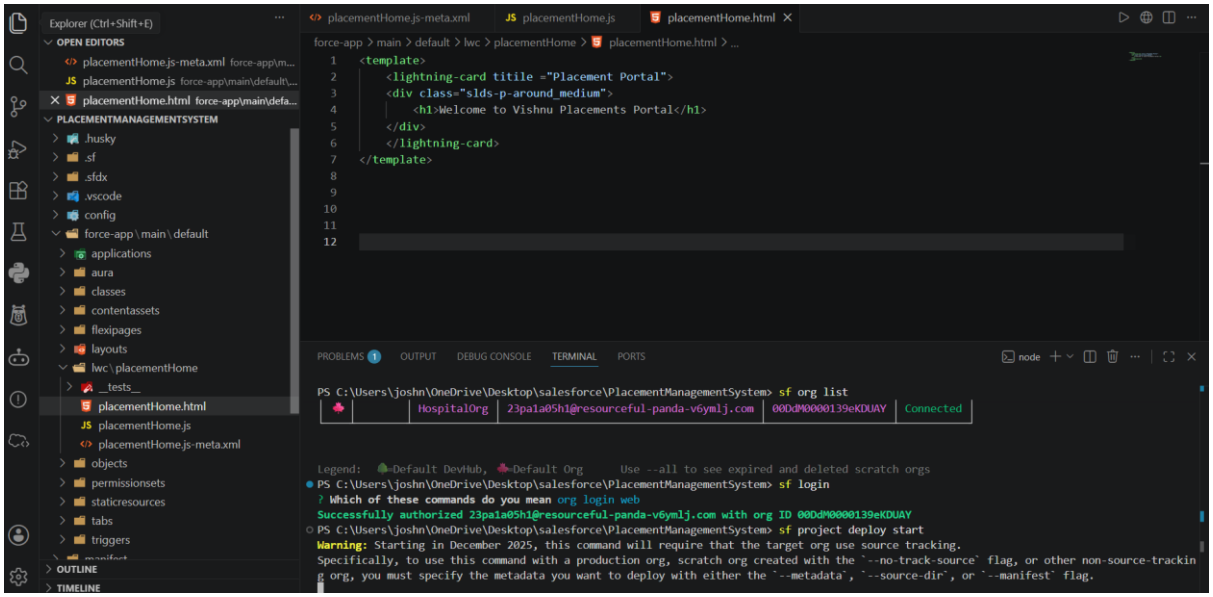
Part 3 – Hands-on Activity 1

In the hands on experience the activity 1 I have created the first lightning component called placementHome .

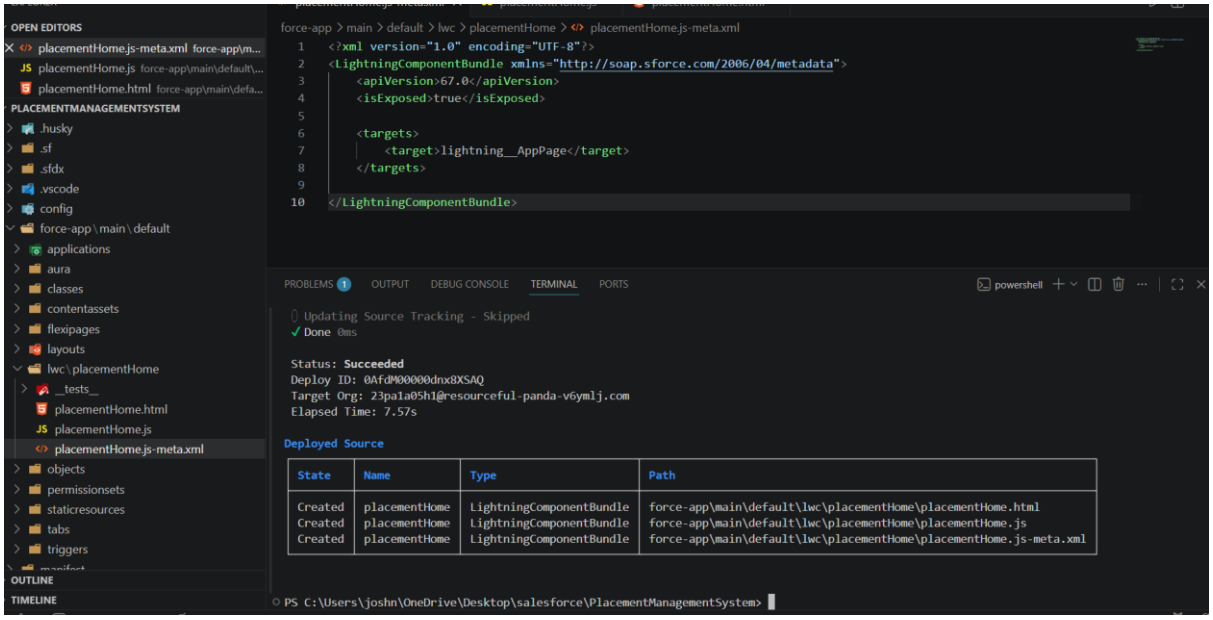
Before that I have created the new project called placementmanagemantsystem that shows the placement of the each students

Basically I have created the component called the placementhome then the it will create the html , js, xml has been created automatically to it .

In the html page we need to display the message called the **"Welcome to Vishnu Placement Portal"**.

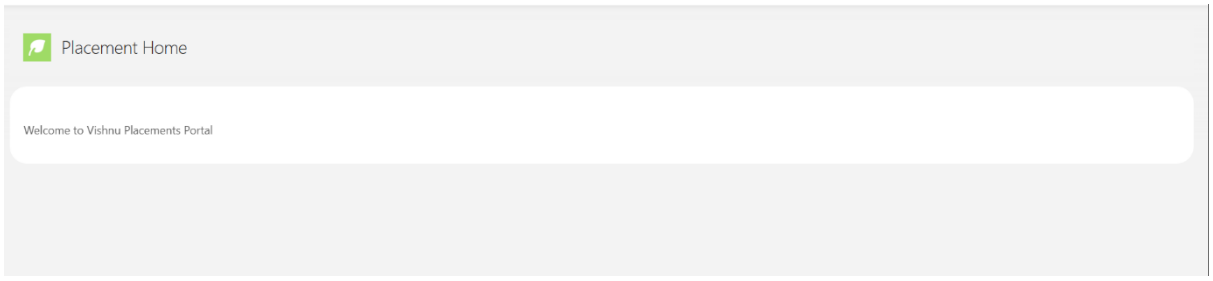


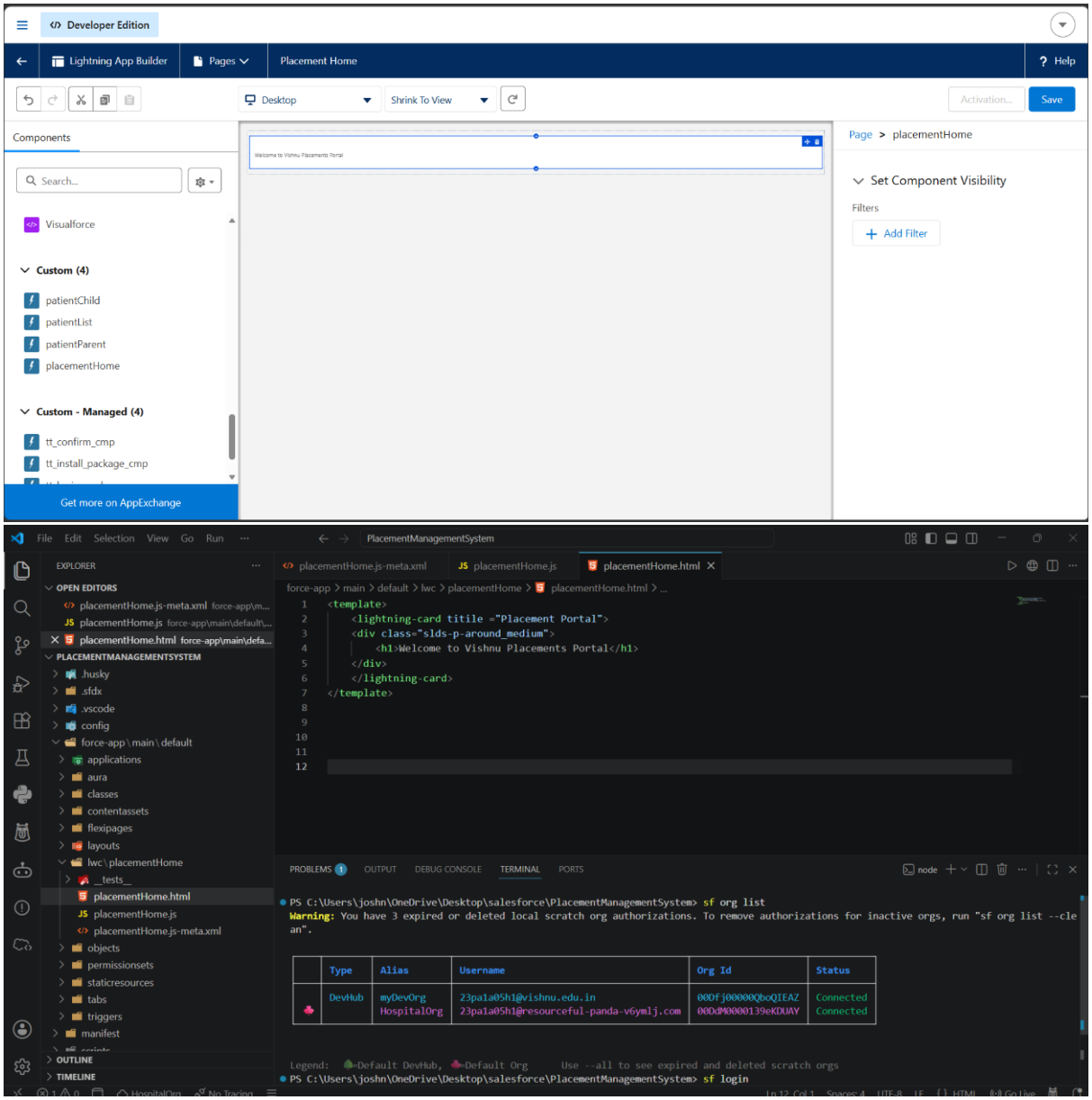
For the xml I have created the lightning app builder that only displays in the app



Then I have deployed of then in the app lanchche I have searched for the lighting app builder then we have to create the new one of the app apge of it

The placementhome has shown in the custom componets then drag into the main page.





The component displayed a welcome message inside a Salesforce Lightning Card, confirming successful deployment

---

Part 4 – Hands-on Activity 2

In this we have Created the JavaScript variables:

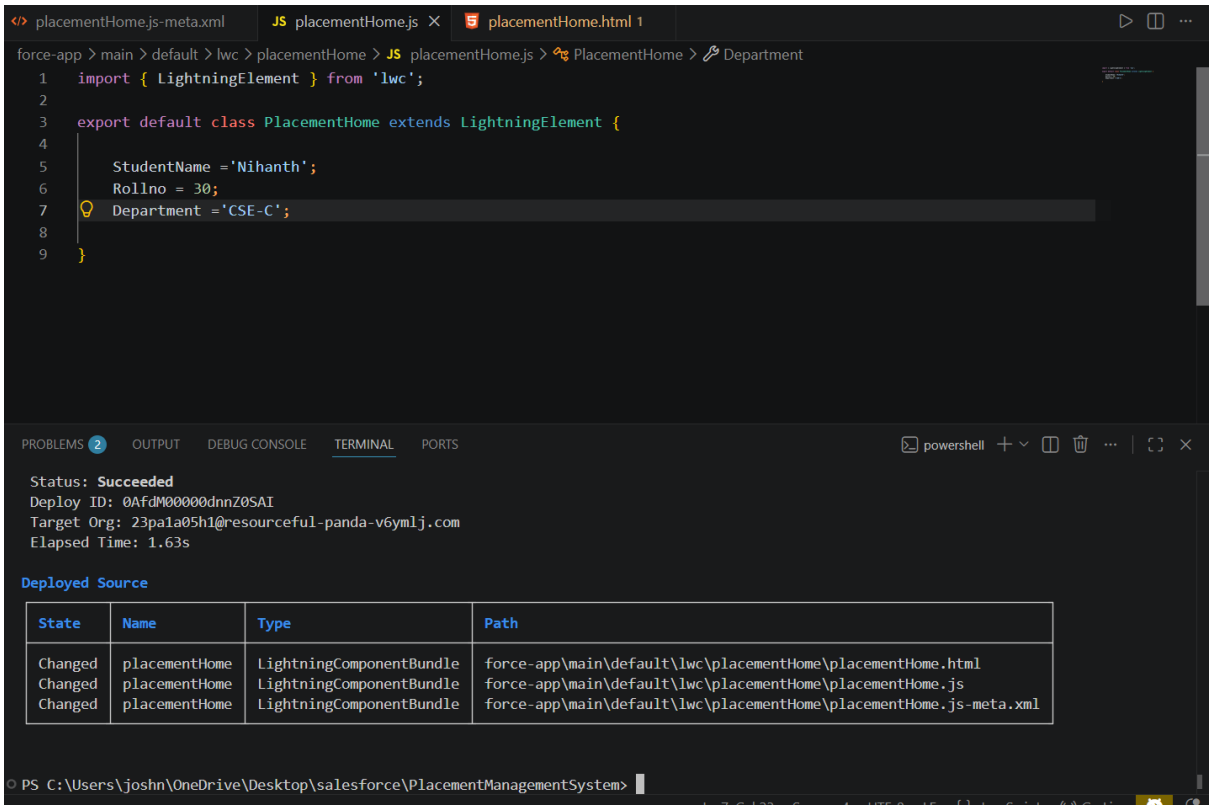
- Student Name
- Roll Number
- Department

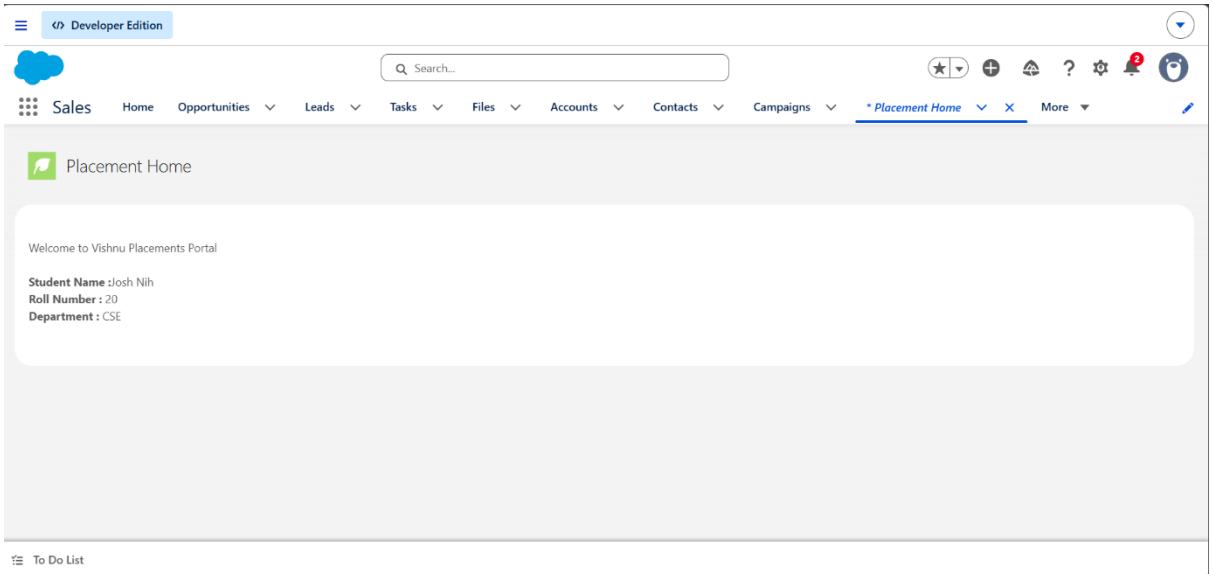
Displayed the variable values using data binding expressions inside the HTML file.

Example:

- Student Name: Koushiik
- Roll Number: 22B81A05H7
- Department: CSE

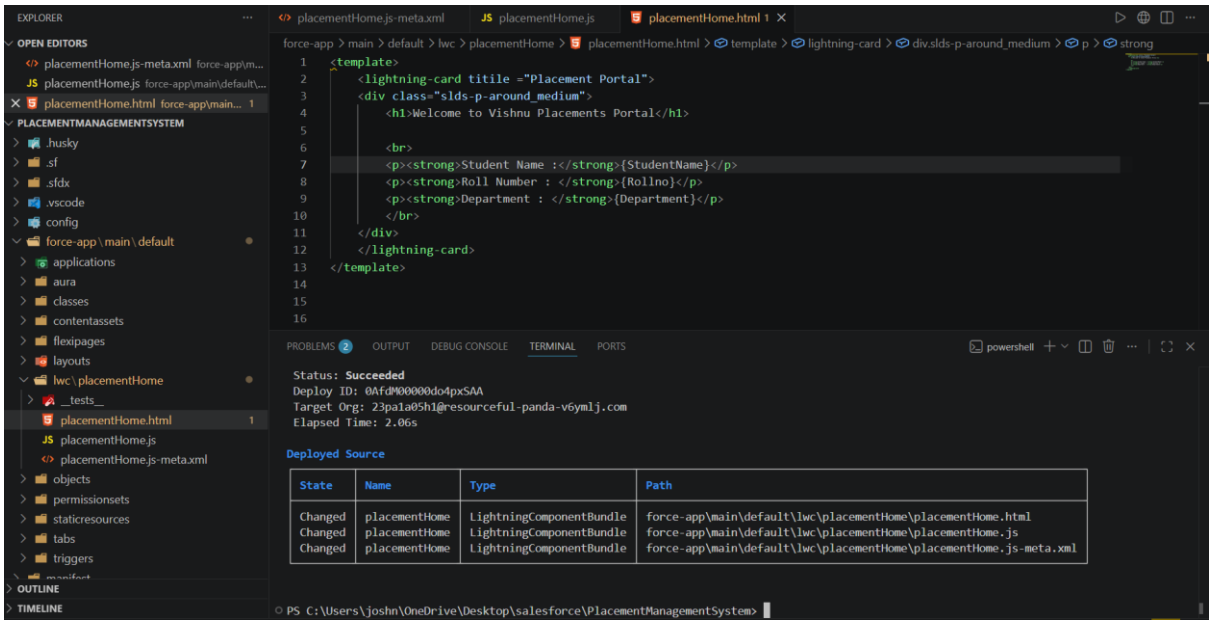
Modified the variable values in the JavaScript file and observed the changes reflected automatically in the user interface.

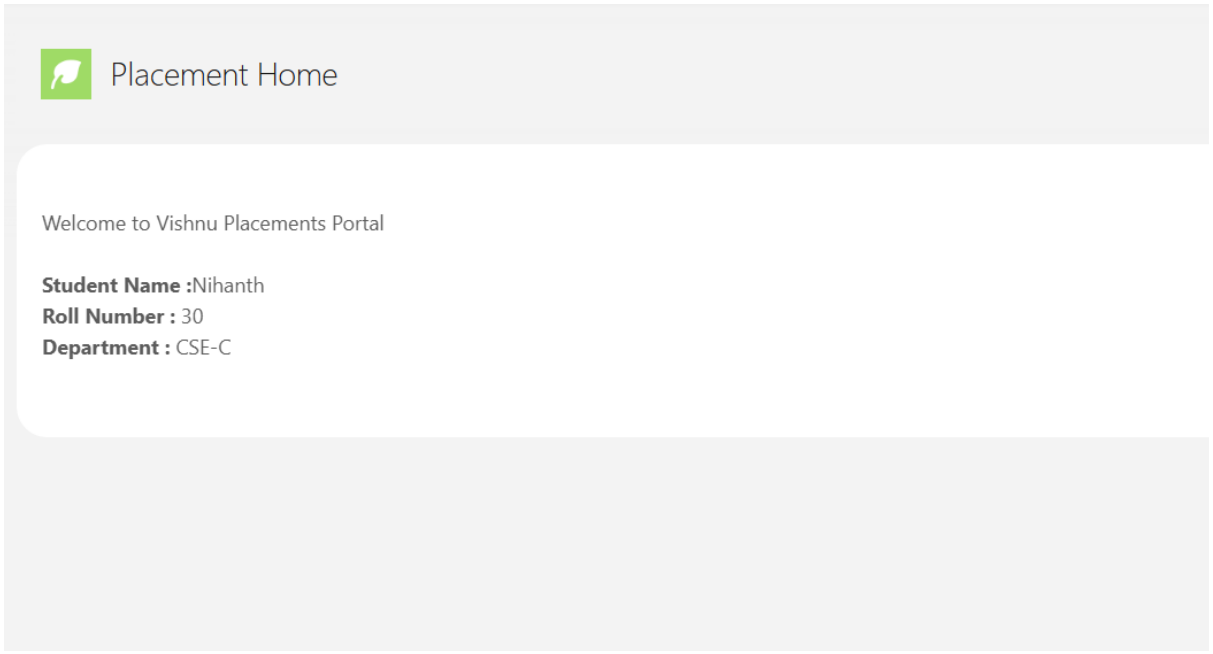




While changing the name in the html code the it gives the perfect name but for that we need to deploy it again for that then we will verify it in the placementshome

In the app lancher we can check wheather the data is changing or not.





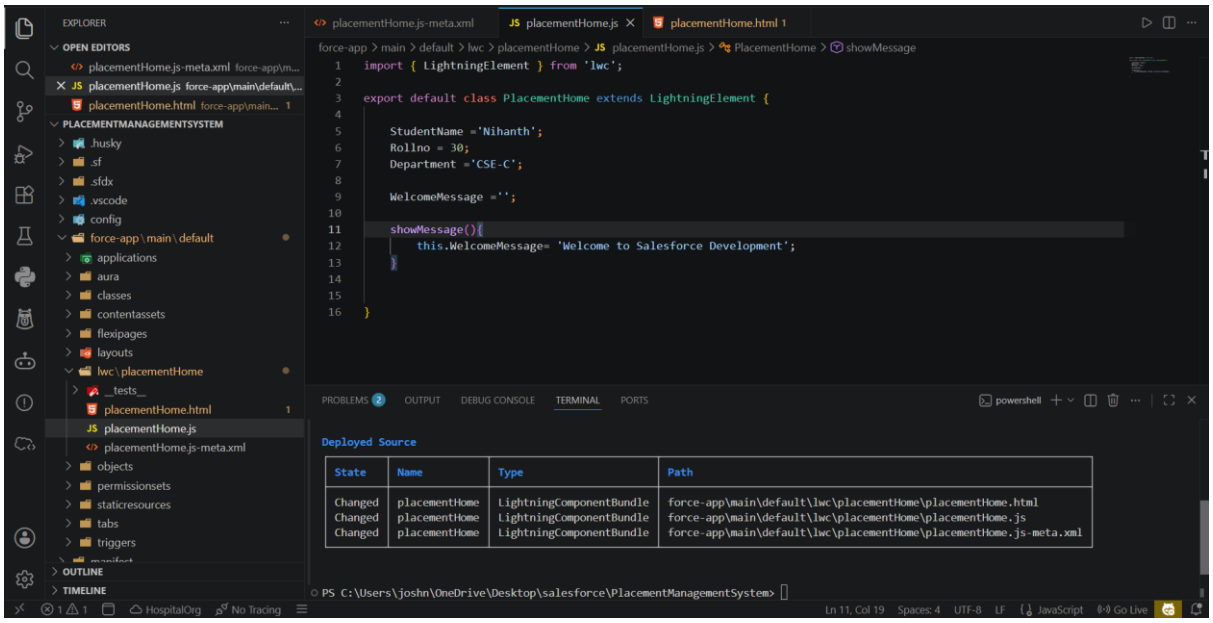
The exercise provided a practical understanding of data binding, one of the core concepts of Lightning Web Components.

Part 5 – Hands-on Activity 3

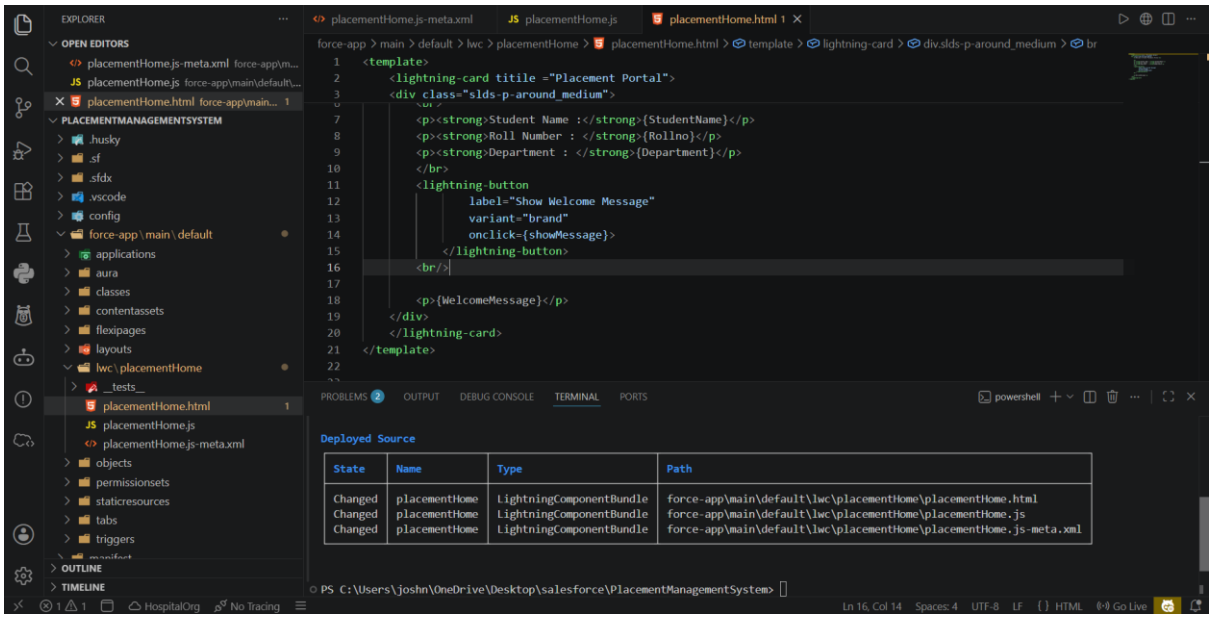
To implement event handling by displaying a welcome message

when a button is clicked.

Created a Lightning Button labeled "Show Welcome Message".



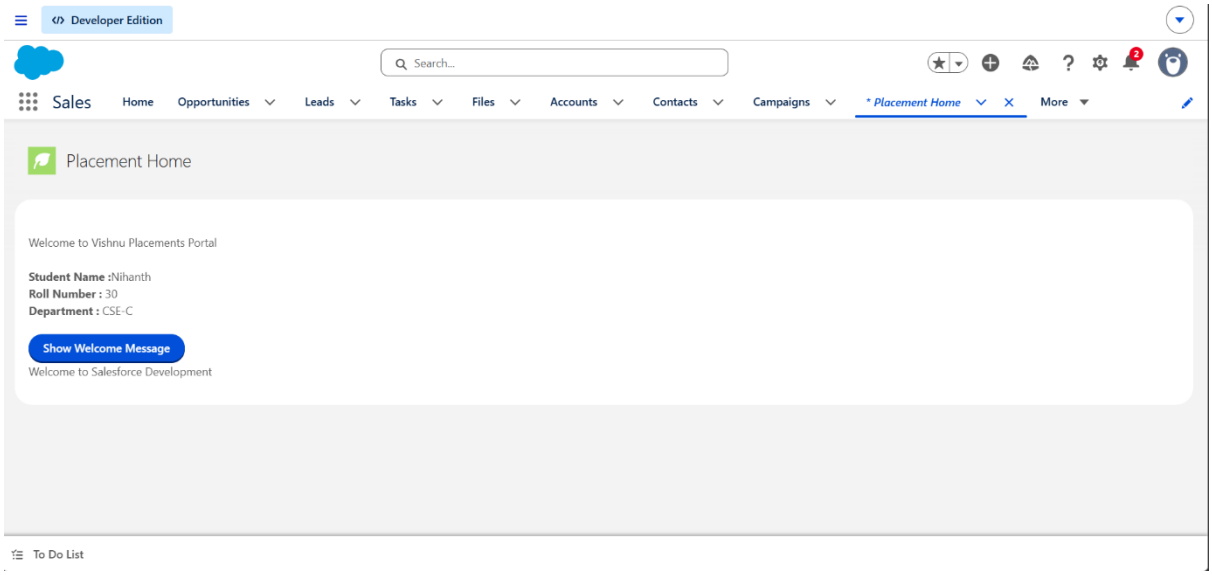
Implemented a JavaScript method named showMessage().



Connected the button to the JavaScript method using the onclick event.

Displayed the message **"Welcome to Salesforce Development."** after the button was clicked.

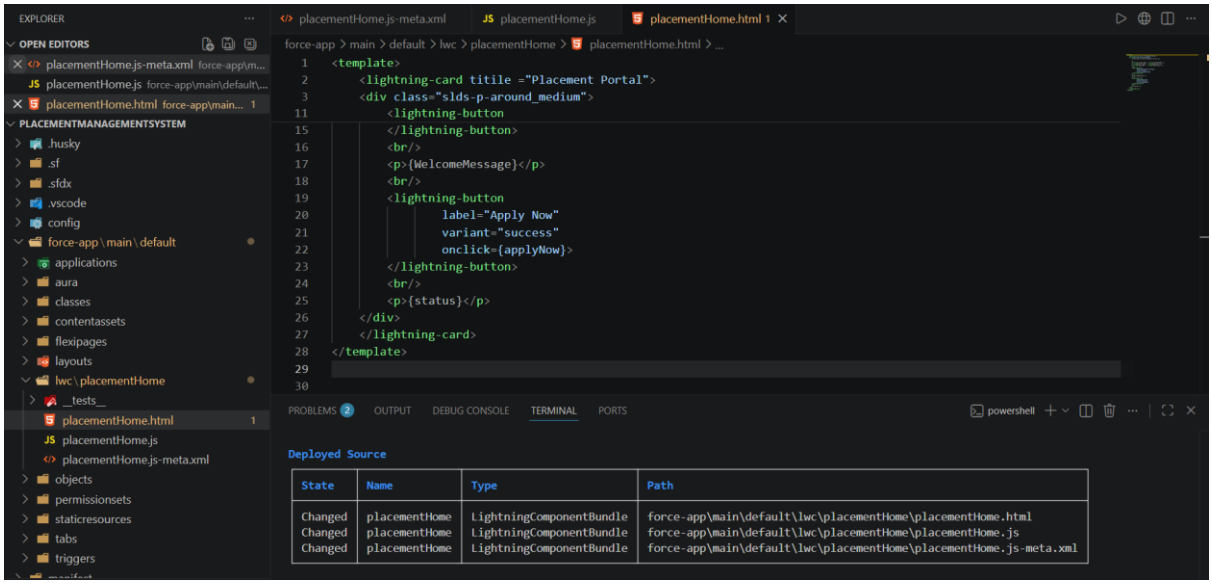
For every time we need to deploy it.



The implementation successfully demonstrated event-driven programming in Lightning Web Components, improving the interactivity of the application.

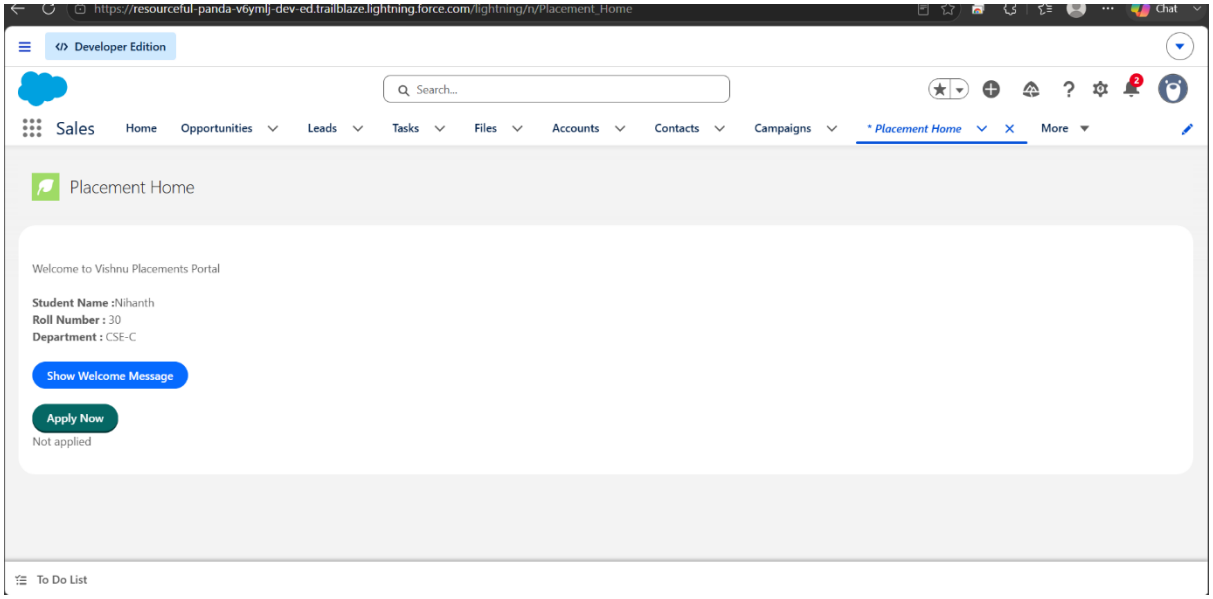
Part 6 – Hands-on Activity 4

To implement dynamic status updates using JavaScript event handling in a Lightning Web Component.

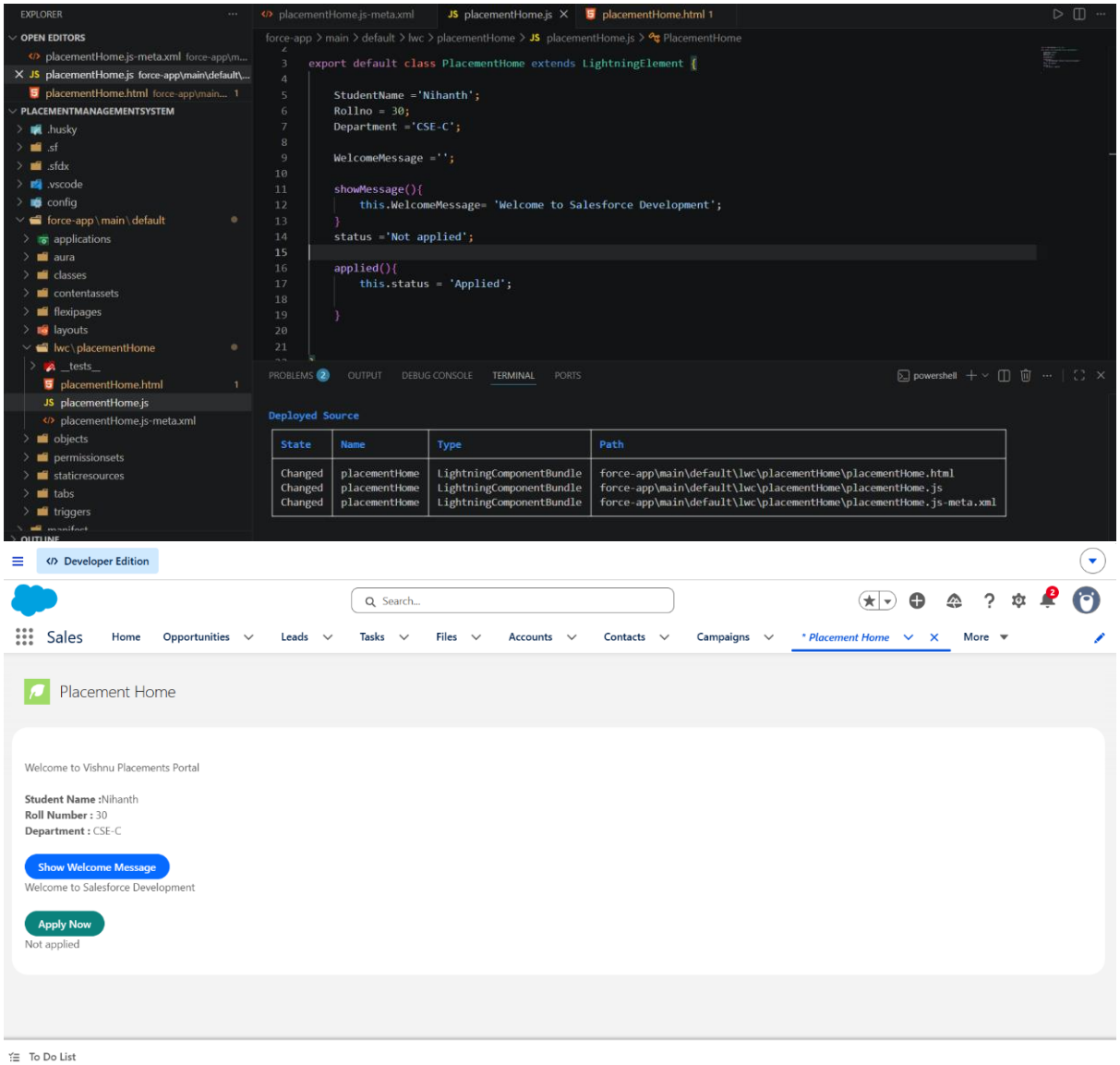


Created a status variable with the initial value **"Not Applied"**.

Added an **Apply Now** button to the user interface.



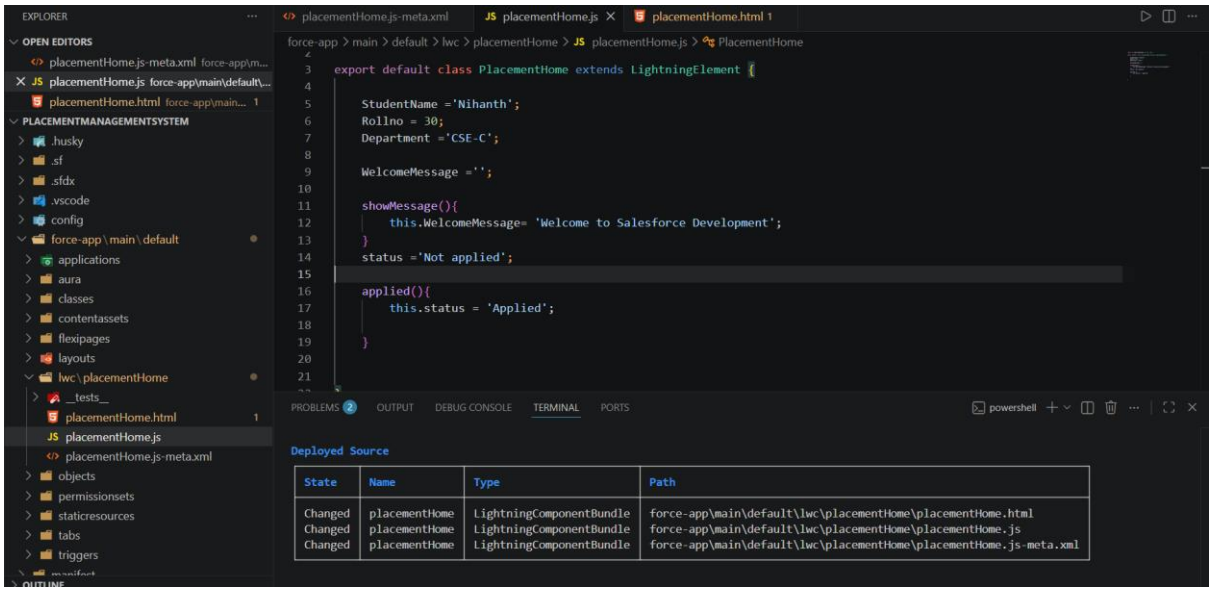
Implemented the applyNow() method to update the status to **"Applied"** when the button is clicked.



This activity demonstrated how LWC responds to user interactions by updating component data without using Apex or a database.

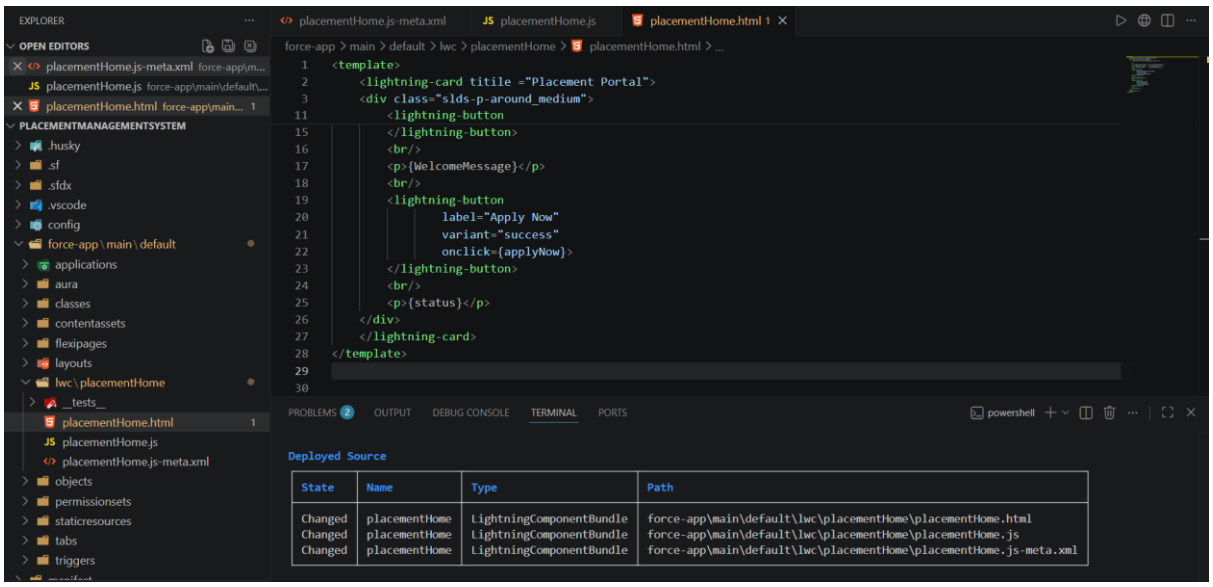
Part 7 – Mini Project Enhancement

To build the first screen of the Placement Management System using Lightning Web Components.



Designed a Placement Management dashboard.

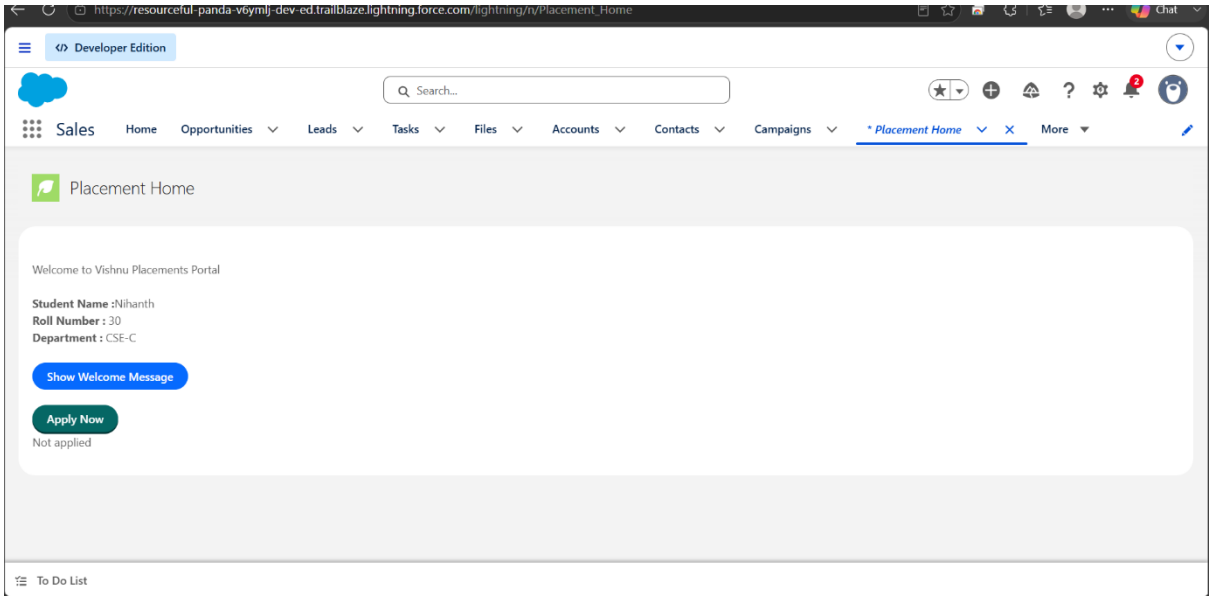
Placement Portal Today's Date Welcome Student Number of Companies : 25 Number of Jobs : 63 Applications Submitted : 5

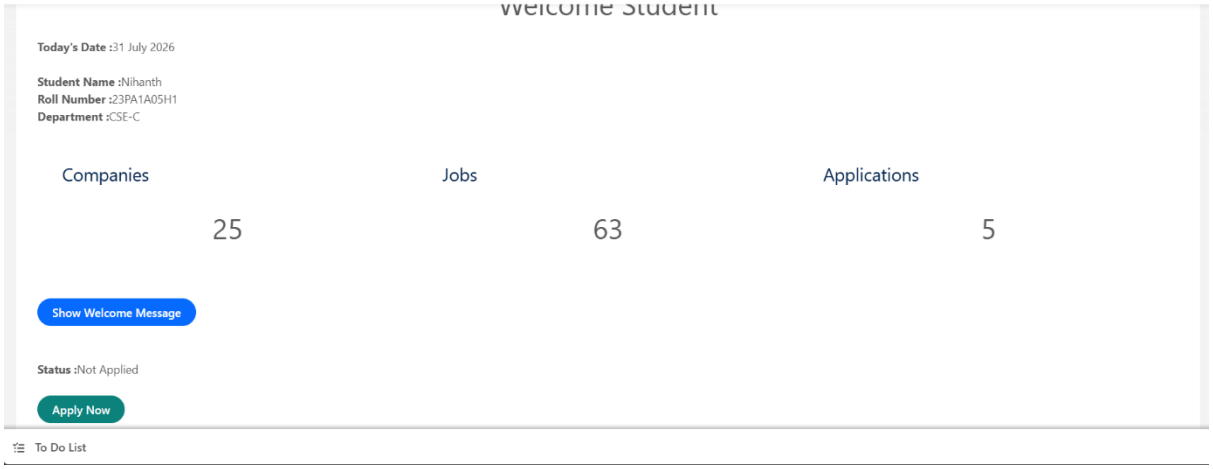


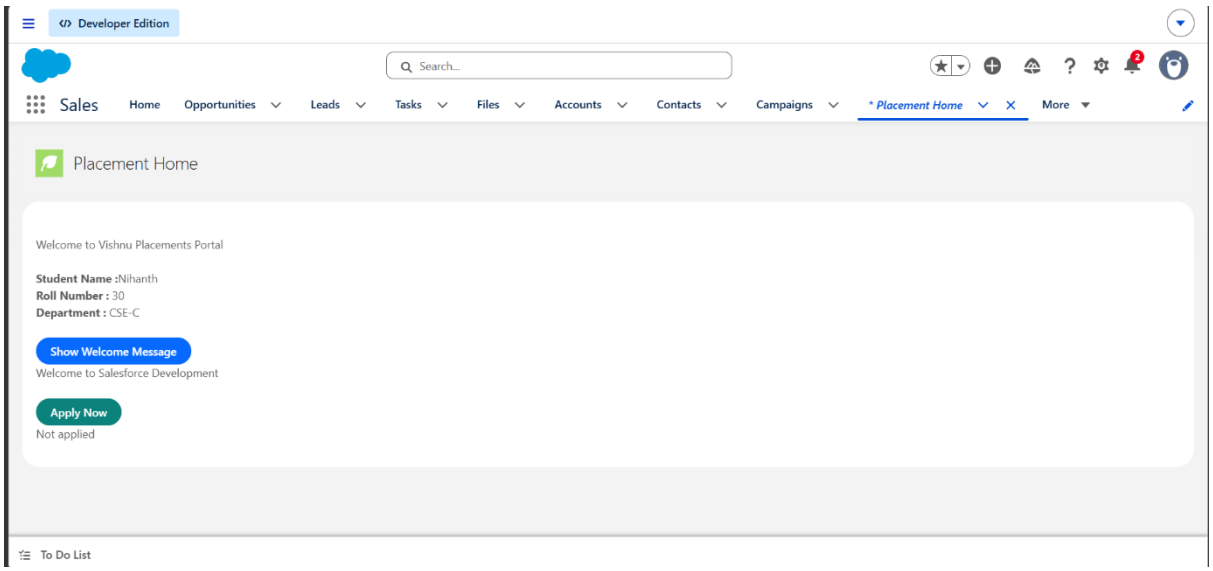
Included interactive buttons for displaying a welcome message and updating the application status.

Displayed the student's information and today's date.

Added dashboard cards for Companies, Jobs, and Applications





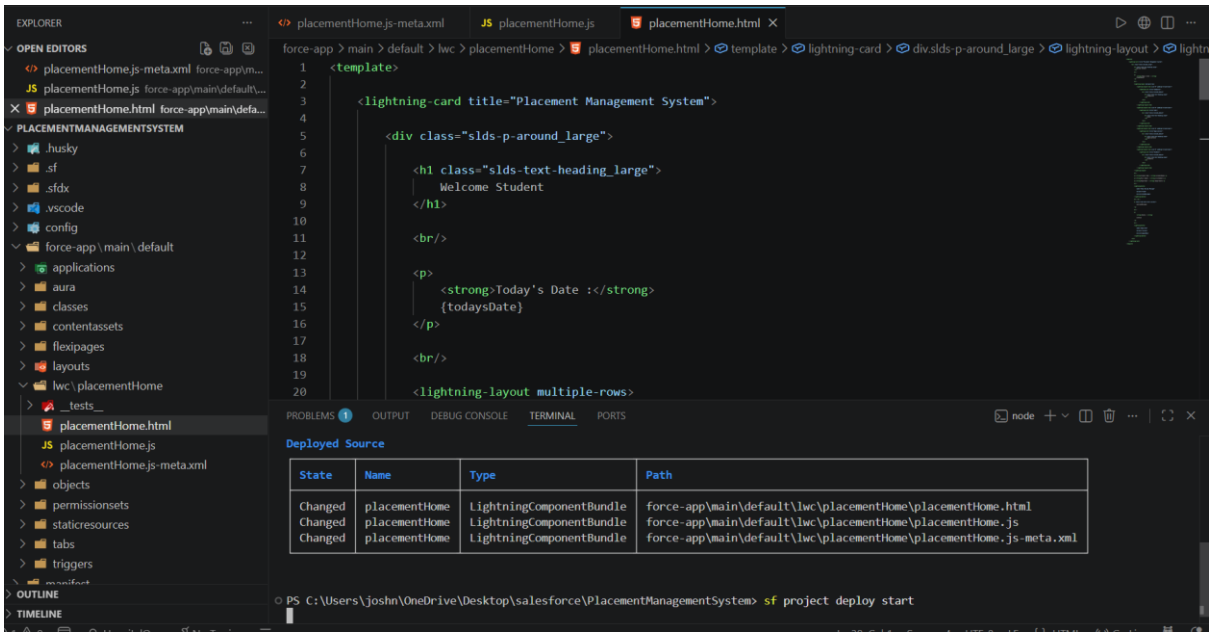


The mini project demonstrated how different LWC concepts can be integrated to build a simple and effective Salesforce application.

Part 8 – Understanding Data Binding

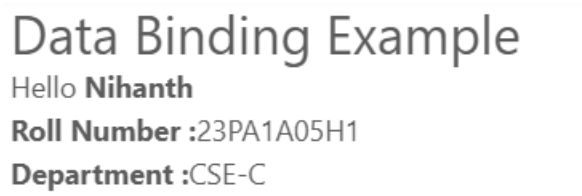
To understand how JavaScript variables are connected to HTML using data binding.

Data Binding is the process of displaying JavaScript variable values in the HTML using curly braces {}. When the variable changes, the updated value is automatically reflected in the user interface.



Displayed the student's name using the {studentName} expression.

Modified the variable value in the JavaScript file.



Observed that the updated value was reflected automatically in the HTML after deployment.

This activity provided a clear understanding of dynamic data display using data binding in Lightning Web Components.

Part 9 – Interview Questions

Q1. What is Lightning Web Components (LWC)?

Lightning Web Components is Salesforce's modern framework for building user interfaces using HTML, JavaScript, and CSS. It is based on web standards and is used to create fast, reusable, and interactive Salesforce applications.

Q2. Why did Salesforce introduce LWC?

Salesforce introduced LWC to improve application performance and simplify development. It uses standard web technologies, making components faster, easier to maintain, and more efficient than Aura Components.

Q3. What is the difference between LWC and Aura Components?

Lightning Web Components (LWC) building user interfaces in Salesforce. Aura Components were introduced first and use Salesforce's proprietary Aura framework, whereas Lightning Web Components are built on modern web standards such as HTML, JavaScript, and CSS

LWC is the newer and recommended framework because it is faster, uses standard web technologies, and provides better performance compared to Aura Components.

Q4. What are the three main files in an LWC?

Every Lightning Web Component contains three essential files:

Every Lightning Web Component consists of three main files: the **HTML** file for designing the user interface, the **JavaScript** file for implementing logic and handling events, and the **Meta XML** file for configuring the component and making it available in Lightning App Builder.

Q5. Why is JavaScript required in LWC?

JavaScript is used to implement the component's logic. It stores variables, handles button clicks, processes user input, and updates the user interface dynamically.

Q6. What is Data Binding?

Data Binding is the process of displaying JavaScript variable values in the HTML using curly braces {}. When the variable changes, the updated value is automatically reflected in the user interface.

Example

JavaScript:

studentName = 'Rahul';

HTML:

Hello {studentName}

Output:

Hello Rahul

Q7. Can LWC directly execute SOQL?

No, LWC cannot execute SOQL directly because SOQL is a server-side operation. Instead, LWC calls an Apex class, and the Apex class executes the SOQL query.

Q8. Why does LWC need Apex?

LWC uses Apex to perform server-side operations such as retrieving records, inserting data, updating records, deleting records, and executing SOQL queries securely.

Q9. Where is an LWC deployed?

Answer

An LWC is deployed to Salesforce and added to pages using Lightning App Builder. It can be placed on:

Lightning app page

Home page

Record page

depending on the targets defined in the .js-meta.xml file.

Q10. Explain the component you built today.

Today, I developed a Lightning Web Component named placementHome for a Placement Management System. The component displays student information, today's date, placement statistics, and application status.

GITHUB LINK : salesforce-training/Salesforce crashcourse/DAY04/README.md at main · joshnandam/salesforce-training